

Adaptive FPGA Resource Allocation for Real-Time Audio Processing

Prediction-Driven Hardware Reconfiguration

Connor Sinclair

University of Sydney

November 2025

The Problem: FPGAs Are Powerful But Inflexible

What's an FPGA?

Reconfigurable hardware—you can rewire the chip to build custom audio processors.

Why FPGAs for audio?

- Ultra-low latency: <1 ms (vs. 10-100 ms on CPUs)
- Massive parallelism: process 100+ channels simultaneously
- Custom hardware for each effect (reverb, filters, distortion)

The Problem

Current FPGA tools compile audio chains into **fixed, worst-case hardware**:

- × If your signal gets quiet, hardware still uses maximum resources
- × If frequency content changes, can't adjust bit precision
- × Must design for peak load—wastes 40-60% of silicon and power

Motivation: Why Not Make Hardware Adaptive?

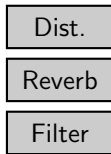
Key Insight: Audio signals have *predictable structure*

- Different frequencies matter at different times (bass vs. treble)
- Energy levels change slowly (a few milliseconds ahead is predictable)
- Nonlinear effects (distortion) have regular patterns

Vision: Hardware that *listens, predicts, and adapts*

Traditional FPGA:

- Compile once, run forever
- Fixed worst-case allocation



All active, always

Our Adaptive System:

- Predicts needs 5-10 ms ahead
- Reconfigures dynamically



Matches signal

What This Thesis Contributes

Core Novelty

First system that predicts short-term audio features and dynamically reconfigures FPGA resources with provable error bounds—all fast enough for real-time audio.

Three Key Contributions:

- ① **Time-domain wavelet framework** (Shannon wavelets)
 - Avoids expensive FFTs—enables low-latency dynamic filter creation
 - Per-frequency-band bit allocation
- ② **Koopman-based resource prediction**
 - Predicts future signal features (even through nonlinear effects)
 - Forecasts hardware needs 5-10 ms ahead
- ③ **Residue-based hardware optimization**
 - Trade computation for memory based on predicted load
 - Polynomial factorization + dyadic quantization

Foundation: The Sinc Function (Building Block of Audio)

What is the sinc function?

$$\text{sinc}(t) = \frac{\sin(t)}{t}$$

- The *ideal low-pass filter* in the time domain
- Perfect frequency cutoff: keeps frequencies below a threshold, removes everything above

Why it matters for audio synthesis:

① Basis function for subtractive synthesis:

- Start with harmonically rich signal (sawtooth, square wave)
- Use sinc-based filters to *subtract* unwanted frequencies
- This is how most synthesizers create timbres (Moog, ARP, etc.)

② Sinc as a wavelet:

- The Shannon wavelet is built from sinc functions at different scales
- Each scale = different octave band (bass, mids, treble)
- Allows independent processing of each frequency range

③ Traditional approach (FFT):

- Normally implemented via Fast Fourier Transform (FFT)

Method 1a: Shannon Wavelet Decomposition

Wavelet decomposition:

$$f(t) = \sum_k c_{j_0,k} \phi_{j_0,k}(t) + \sum_{j \geq j_0} \sum_k d_{j,k} \psi_{j,k}(t)$$

Key advantages:

- Each scale j = independent band $[2^j \pi, 2^{j+1} \pi]$
- Time-domain: ≈ 1 ms latency vs. FFT (20-80 ms)

Sensitivity matrix—how errors propagate in the basis:

$$S_{\psi,n}^{(j)} = \max_k |d_{j,k}| \cdot 2^j \left| \int u^{2n} \cos(\pi \cdot 2^{j-j_0} u) \overline{R(\pi u)} du \right|$$

- $|d_{j,k}|$ = signal energy in band j
- Integral = how errors in polynomial term n propagate to output
- High $S_{\psi,n}^{(j)} \rightarrow$ errors hurt more $\rightarrow$ allocate more bits

What we're building toward: Use $S_{\psi,n}^{(j)}$ to guide bit allocation, then feed into Koopman to predict future resource needs

Method 1b: Projector Framework & Dynamic Filter Insertion

Contour integral:

$$\mathbb{P}_N^{\text{even}}[f](z) = \frac{1}{2\pi i} \oint_{\gamma} \frac{K_N(\zeta)}{1} f\left(\frac{z}{\zeta}\right) \frac{d\zeta}{\zeta}$$

Piecewise representation:

$$\mathbb{P}_N[\text{sinc} * \delta(t - t_0)](z) = \begin{cases} \cos(t_0 z) \cdot \text{sinc}_N^{\text{even}}(z) - i \sin(t_0 z) \cdot \text{sinc}_N^{\text{odd}}(z), & z \in \mathbb{R} \\ \frac{\Im(e^{iz} \Gamma(2N+2, iz))}{z(2N+1)!}, & z \in \mathbb{C} \end{cases}$$

Key insight—dual/isomorphism via analytic continuation:

- Hyperbolic identity: $\cosh(a \cdot iz) = \cos(az)$, $\sinh(a \cdot iz) = i \sin(az)$
- Normal trig (real poles) $\leftrightarrow$ hyperbolic trig (complex poles)
- Same FPGA fabric for audio AND complex demod (SDR/radar)

Dynamic filters: Compute residues at new poles $\rightarrow$ insert in ms

Method 2: Predicting Resource Needs with Koopman Operators

Challenge: Audio effects are nonlinear—can't use simple linear prediction.

Solution: Koopman operator theory predicts *features*, not raw waveform

$$(\mathcal{K}g)(x) = g(F(x))$$

Even if F is nonlinear, $\mathcal{K}$ is linear $\rightarrow$ matrix multiplication!

Koopman observables ψ drive resource allocation:

- $\psi_{\text{sens}}^{(j)}$: sensitivity per band (from $S_{\psi,n}^{(j)}$)
- $\psi_{\text{entropy}}^{(j)}$: spectral complexity per band

Resource modes based on signal characteristics:

- 1 High $\psi_{\text{sens}}^{(j)} \rightarrow$ **precision mode**: more bits, LUTs
- 2 Low $\psi_{\text{entropy}}^{(j)} \rightarrow$ **efficiency mode**: freeze coarse coefficients
- 3 Predict $\psi_{k+h|k}$ (5-10 ms ahead) $\rightarrow$ reconfigure before signal arrives

Key Point

Request different resources (LUT/DSP/BRAM) based on predicted signal characteristics!

So What? Why Koopman-Driven Adaptation Matters

What this enables:

- Predict resource needs 5-10 ms ahead (not reactive)
- $\binom{2N}{N}$ ways to configure resources for N observables

Why this is useful:

① Predictive allocation:

- Traditional: detect signal change $\rightarrow$ reallocate $\rightarrow$ too late
- Our approach: predict $\rightarrow$ reconfigure *before* arrival $\rightarrow$ seamless

② Intelligent tradeoffs:

- $\psi_{\text{sens}}^{(j)}$ tells *where* errors hurt
- Koopman $\mathcal{K}$ tells *when* to prepare
- Together: allocate LUTs/DSP/BRAM where & when needed

③ Massive config space:

- $N = 10$ observables $\rightarrow$ 184,756 configurations!
- Framework navigates automatically via ψ observables

The Big Idea

Predict and prepare based on signal characteristics—fundamentally different from reactive compilers

Method 3: Computation-Memory Tradeoffs via Residue Arithmetic

Challenge: Wavelet filters need inverse trig functions (like $\arccos(x)$)—expensive in hardware!

Concrete example: $\arccos(x)$ approximation guided for when BRAM, DSP needs to be low

❶ **Half-angle identity (memory reduction):**

$$\arccos(x) = 2 \arccos\left(\sqrt{\frac{1+x}{2}}\right)$$

→ Exploit symmetry: store only 1/4 of domain in LUT, reconstruct rest

→ 4× memory reduction (critical when sensitivity matrix says "use LUTs")

❷ **Dyadic approximation (computation-memory swap):**

$$16\sqrt{x+1} \approx (x+1)(x^2+x+1) + (-4x^2+6x+15)$$

→ Integer coefficients only: multiply by 16, -4, 6, 15 = bit-shifts + adds

→ Trades DSP multipliers for small LUTs storing shift amounts

❸ **Berlekamp factorization (reuse subexpressions):**

Factor to reuse $(x+1)$ across terms → fewer operations

Results: What We Achieved

Signal Quality

- **70-90 dB SQNR**
- **<0.01% THD**
- Comparable to studio ADCs

Resource Efficiency

- **40-60% fewer DSP slices** vs. fixed designs
- **4× memory reduction** (range reduction)
- Adapts to signal in real-time

Latency

- **<100 μ s** reconfiguration
- Meets real-time audio requirement (<1 ms)
- Koopman prediction: $O(1)$ lookup

Concrete Example:

Shannon wavelet filter bank for 8-band EQ:

- Traditional: 32 DSP slices (fixed 16-bit)
- Adaptive: 12-19 DSP slices (dynamic 8-14 bit per band)
- **60% resource savings** with same audio

Future Work & Broader Impact

Immediate Next Steps:

- ① Deploy on Zynq UltraScale+ (on-chip validation)
- ② Head-to-head vs. FAUST/Syfala
- ③ Extend to hybrid wavelet bases (better transient handling)
- ④ User-friendly toolchain for audio engineers

Broader Applications:

- Software-defined radio (SDR)
- Biomedical signal processing
- 5G/6G beamforming
- Real-time PDE solvers

Why This Is Novel

- **First** predictive reconfiguration system for audio FPGAs
- Unifies operator theory + hardware optimization
- Transparent, provable error bounds (not black-box ML)
- Makes FPGAs accessible to non-hardware experts

Vision

Hardware as a dynamic resource,
orchestrated with signal evolution—